

Universe Reinsurance Platform — Database Link & Alignment Checklist

Run after every deploy or repository switch. Extracted from the *Repository Migration & Database Alignment Guide* v1.0, §7 — the guide holds the full context, seeding rules and troubleshooting.

Version 1.0 — September 2026 — Confidential, for internal IT use only. Server layout per the v2.0 specification: `/opt/universe`, user `universe`, `/opt/universe/.env`, PM2, nginx, PostgreSQL on localhost. Source: https://github.com/iwadaya/Saudi_re.

Checklist — is the database linked to the app, and does it align?

Work through the groups in order. "Command" runs on the application server; `psql` commands need `DATABASE_URL` in the shell — load it once with `cd /opt/universe && set -a && . /.env && set +a` (as `universe` or root) **in a shell you will not use for `pm2` commands** (see P1). Tick every row before sign-off.

Deployment-kit (systemd) server? Use these variants: C1 → `ls -lL /opt/universe/.env = -rw-r---` root universe (a symlink to `/etc/universe/universe.env`); C6, R1, R2 → `sudo journalctl -u universe --no-pager -n 300 | grep ...` instead of `pm2 logs`; C7 → `sudo systemctl show universe -p Environment` (should be empty apart from what the unit file sets); P1 → one process, `pool.max 50` unless `DB_POOL_MAX` is set in the env file; P2 unchanged.

Boot lines scroll away. C6, R1 and R2 read lines written at start; with four workers and normal traffic they leave the last 300 `pm2 logs` lines within minutes (every request logs a line). Run them right after the reload, or `grep` the merged log file, which keeps everything: `grep 'DB pool connecting' /opt/universe/logs/pm2-out.log | tail -n 4` (same for `startup configuration loaded` and R2's patterns).

7.1 Configuration — the app is told about exactly one database

#	Check	Command	Expected	Proves
C1	One env file, locked down	<code>ls -l /opt/universe/.env</code>	<code>-rw----- universe universe</code>	Secrets readable by the app user only
C2	Exactly one <code>DATABASE_URL</code>	<code>grep -c '^DATABASE_URL=' /opt/universe/.env</code>	1	No duplicate/overridden connection string
C3	No placeholders	<code>grep -c CHANGE_ME /opt/universe/.env</code>	0 (grep exits 1 when the count is 0 — expected)	Real values everywhere
C4	Production mode, migrations explicit	<code>grep -e '^NODE_ENV=' -e '^RUN_MIGRATIONS_ON_BOOT=' /opt/universe/.env</code>	<code>NODE_ENV=production , RUN_MIGRATIONS_ON_BOOT=false</code> (or absent)	Schema changes only happen in the deploy step

#	Check	Command	Expected	Proves
C5	Seeding posture	<code>grep '^SEED_ON_DEPLOY=' /opt/universe/.env</code>	no output (absent; grep exits 1), or exactly <code>SEED_ON_DEPLOY=reference (ref / reference-only are synonyms)</code> . Any other value — <code>1</code> , <code>true</code> , <code>yes</code> , <code>on</code> , <code>if-empty</code> , <code>reset</code> , <code>always</code> — loads the fabricated treaty portfolio on the next deploy: remove it	No fabricated treaties can be seeded
C6	The app found <i>that</i> file	<code>sudo -u universe -H pm2 logs universe - -lines 300 --nostream 2>/dev/null grep 'startup configuration loaded' tail -n 1</code>	newest line (format <code>0 universe <date>: {json})</code> with <code>"envFile":"/opt/universe/.env"</code>	The file exists where the app looks for it (C7 proves it is the one in effect)
C7	Nothing overrides the file inside the workers	<code>for p in \$(pgrep -u universe -f 'server/src/index.js'); do sudo tr '\0' '\n' < /proc/\$p/environ grep -E '^(DATABASE_URL AUTH_JWT_SECRET SESSION _SECRET SEED_ON_DEPLOY PORT RUN_MIGRATI ONS_ON_BOOT DB_POOL_MAX)= ' sed -E 's# (://[^\:]+\:)[^\@]*@#\1***@#'; done and sudo grep -c '"DATABASE_URL"' /home/universe/.pm2/dump.pm2</code>	exactly one <code>DB_POOL_MAX=<n></code> line per worker (injected by <code>ecosystem.config.js</code>) and nothing else; <code>0</code> from the dump file. PM2 copies the <i>whole shell environment</i> of the <code>pm2 start / pm2 reload</code> command into the workers, and <code>dotenv</code> never overrides a variable that is already set — so a shell that had sourced <code>.env</code> (or exported <code>DATABASE_URL</code> for a backup) bakes those values in, and <code>pm2 save</code> carries them across reboots; every later <code>.env</code> edit is then ignored. Fix: reload from a clean environment — <code>sudo -u universe -H env -i HOME=/home/universe PATH="\$PATH" bash - c 'cd /opt/universe && pm2 reload ecosystem.config.js && pm2 save'</code>	The workers take these values from <code>/opt/universe/.env</code> , and will again after a reboot

#	Check	Command	Expected	Proves
C8	No second env file	<code>sudo ls -la /opt/universe/server/.env</code>	No such file or directory	The <code>--prefix</code> server scripts (migrate, seed, <code>manage-user.js</code>) and the app read the same file (§4 Step 4)

7.2 Connectivity — the database answers with the app's own credentials

#	Check	Command	Expected	Proves
D1	Connect with the app's URL	<code>psql "\$DATABASE_URL" -c 'select 1'</code>	one row, 1	Host, port, role, password and database name are right
D2	Identify what you are connected to	<code>psql "\$DATABASE_URL" -tAc "select current_database(), current_user, inet_server_addr(), inet_server_port(), version()"</code>	<code>universe universe 127.0.0.1 5432 PostgreSQL 16...</code> (your values)	You are looking at the intended database, not a stray one
D3	Postgres is up and listening locally	<code>sudo systemctl status postgresql --no-pager head -3</code> and <code>ss -tlnp grep 5432</code> (alternatives: <code>pg_lsclusters</code> ; <code>psql "\$DATABASE_URL" -tAc "show listen_addresses"</code>)	<code>active (running)</code> ; only loopback sockets — <code>127.0.0.1:5432</code> and/or <code>:::5432</code> , never <code>0.0.0.0:5432</code> , <code>*:5432</code> or <code>:::5432</code> (<code>pg_lsclusters</code> → <code>online</code> ; <code>listen_addresses</code> → <code>localhost</code>)	Database service healthy and not exposed
D4	Role owns the schema	<code>psql "\$DATABASE_URL" -tAc "select pg_get_userbyid(datdba) from pg_database where datname=current_database()"</code>	<code>universe</code>	Migrations can create/alter objects without superuser help

7.3 Runtime — the *running* application is connected to *that* database

#	Check	Command	Expected	Proves
R1	Boot log names the database	<code>sudo -u universe -H pm2 logs universe -l -lines 300 --nostream 2>/dev/null grep 'DB pool connecting' tail -n 1</code>	<code>"url":"postgresql://universe:***@localhost:5432/universe"</code> matching <code>DATABASE_URL</code> (password masked)	The process is using the same host/port/db you tested in D1–D2

#	Check	Command	Expected	Proves
R2	Boot sequence completed	same log, <code>grep -e 'database connection verified' -e 'migrations skipped' -e 'migrations complete' -e 'reference data ready' -e 'http server listening' tail -n 12</code>	For the most recent start, once per worker : database connection verified, migrations skipped, reference data ready (appears twice per worker), http server listening. migrations skipped is correct — the deploy applied them; migrations complete appears instead only if <code>RUN_MIGRATIONS_ON_BOOT=true</code>	App verified the DB, found the schema, seeded, and is serving
R3	Deep health via the app port	<code>curl -sS -D - http://127.0.0.1:4000/api/health/deep</code>	HTTP 200, "db": {"ok":true,"pingMs":<small>,"pool":{...,"max":<per-worker pool>,...}}, header X-Pool-Waiting: 0. Under <code>ecosystem.config.js</code> the per-worker pool is <code>floor(80 / workers)</code> — 20 on a 4-worker box, 40 with 2 workers; 50 only when <code>DB_POOL_MAX</code> is set or the app runs outside the ecosystem file (see P1)	Live round-trip from the app to the DB, pool not saturated
R4	Deep health through nginx/TLS	<code>curl -sS https://<APP_DOMAIN>/api/health/deep</code>	same body, "env": "production"	Users' path reaches the same connected app
R5	DB sees the app's connections	<code>psql "\$DATABASE_URL" -c "select username, client_addr, state, count(*) from pg_stat_activity where datname=current_database() group by 1,2,3"</code>	rows with <code>username = universe</code> (idle/active) from the app host	The connections exist in Postgres, from the expected role and host

#	Check	Command	Expected	Proves
R6	Read path: app data = DB data	<pre>curl -sS http://127.0.0.1:4000/api/auth/users grep -o '"username":"[^"]*"' cut - d'"' -f4 sort vs psql "\$DATABASE_URL" -tAc "select username from public.uw_user where is_active order by 1"</pre>	identical lists with the same row count. Beware the static fallback the API serves when <code>uw_user</code> is empty or missing: exactly two rows, <code>chief.underwriter</code> and <code>underwriter1</code> , with <code>user_id</code> <code>00000000-...-0001 / -0002</code> — the same names a migrated database has, so compare the count with <code>select count(*) from public.uw_user where is_active</code>	The login screen is served from this database
R7	Write path: a login lands in the audit log	<pre>Log in once in the browser, then psql "\$DATABASE_URL" -c "select event_type, payload->'actor' ->>'name' as username, actor as user_id, created_at from public.audit_log where event_type='LOGIN' order by created_at desc limit 3"</pre>	top row: <code>username =</code> the account you just used (the <code>actor</code> column holds its user id), <code>created_at =</code> just now	The app writes to this database, and the clock/timezone are sane
R8	Smoke test	<pre>BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh (add SMOKE_USER / SMOKE_PASSWORD to test a real login)</pre>	PASSED	R3–R4 plus the SPA in one exit code. Its user-list step only checks that the list is non-empty, so it does not replace R6

7.4 Schema alignment — the code and the database agree on the migrations

#	Check	Command	Expected	Proves
S1	Nothing pending	<pre>cd /opt/universe && sudo -u universe -H npm run migrate:status --prefix server tail -n 3</pre>	Pending (0):	Every migration file in this checkout has been applied
S2	Counts match	<pre>psql "\$DATABASE_URL" -tAc "select count(*) from public._migrations" vs ls /opt/universe/server/src/db/migrations/ *.sql wc -l</pre>	equal (149 at this commit)	No file applied twice, none missing

#	Check	Command	Expected	Proves
S3	No orphans (DB ahead of code)	<pre>psql "\$DATABASE_URL" -tAc "select filename from public._migrations" sort > /tmp/applied.txt; ls /opt/universe/server/src/db/migrations sort > /tmp/ondisk.txt; comm -23 /tmp/applied.txt /tmp/ondisk.txt</pre>	empty	The database was never migrated by code this checkout does not contain. If a filename prints, the old repository shipped a migration the new one lacks — stop and reconcile before deploying further (docs/migration-audit.md)
S4	No stragglers (code ahead of DB)	<pre>comm -13 /tmp/applied.txt /tmp/ondisk.txt</pre>	empty	Same as S1, file-by-file
S5	Last migration is recent and expected	<pre>psql "\$DATABASE_URL" -c "select filename, applied_at from public._migrations order by applied_at desc limit 5"</pre>	newest = 157_quote_offer_status_checks.sql at this commit, applied_at = your deploy time	The deploy's migration step ran against this database
S6	Core tables answer	<pre>psql "\$DATABASE_URL" -c "select (select count(*) from public.contract) contracts, (select count(*) from public.quote) quotes, (select count(*) from public.claim) claims, (select count(*) from public.uw_user) users"</pre>	your real business counts, unchanged from before the switch	The data survived; the app's tables exist
S7	Recorded migrations really applied	<pre>psql "\$DATABASE_URL" -tAc "select (select count(*) from pg_indexes where schemaname='public' and indexname in ('uq_country_code_active','uq_class_of_business_name_active','uq_brokers_name_active','uq_companies_name_active')) idx150, (select count(*) from pg_constraint where conname in ('quote_status_check','quote_uw_status_check','contract_offer_status_check','quote_offer_status_check')) chk157, (select count(*) from pg_constraint where conname='uw_user_role_id_fkey') fk156, (select count(*) from public.uw_user where username in ('cuo','underwriter','edwin.taruvinga','catho.ba','chongo.nkalamo')) old132"</pre>	4 4 1 0	S1–S5 only prove the bookkeeping (_migrations stores filenames, no checksums). The July-2026 runner on the v2.0 server recorded a file as applied even when it skipped undefined table/object errors, so this checks that objects from 150, 156 and 157 exist and 132's renames happened. Anything else: follow docs/migration-audit.md ; never edit _migrations by hand

7.5 Reference-data alignment — the lookups the app expects are present

#	Check	Command	Expected	Proves
F1	Reference counts	<code>cd /opt/universe && sudo -u universe -H npm run seed:reference:check</code>	headings Reference data (read-only check): / Business data (never written by this script): ; countries ≥75, currencies ≥32, brokers ≥11, reinsurers ≥25, cedants ≥40, treaty types ≥11, classes ≥15, roles ≥9 (the check counts deactivated rows too; only a fresh database shows the exact §5.2 numbers); business rows = yours, unchanged	The reference seed reached this database
F2	Seed-only cedants and reinsurers visible	SQL in §5.2	MEDGULF, SALAMA, SAICO, Allianz Saudi Fransi and AXA Cooperative Insurance listed; reinsurer count query = 4	GCC/MENA extras present — these names come only from the reference seed
F3	No fabricated treaties	<code>psql "\$DATABASE_URL" -tAc "select count(*) from public.contract where import_metadata->>'source'='seed:test- treaties'"</code>	0	The test portfolio was never loaded
F4	UI dropdowns	New treaty screen: Cedant, Currency, Reinsurer, Class of business, Treaty type	populated as in §5.2	The app reads the seeded lookups

7.6 Capacity and users

#	Check	Command	Expected	Proves
P1	Connection budget	<code>psql "\$DATABASE_URL" -tAc "show max_connections" ; PM2 workers from pm2 status ; per-worker pool from R3 pool.max or sudo -u universe -H pm2 env 0 grep DB_POOL_MAX</code>	workers × pool.max ≤ max_connections – 20 (e.g. 4 × 20 = 80 ≤ 80; this 20-connection headroom is what the ecosystem file enforces and supersedes v2.0 §4.2's 30 — see §1). Under PM2 the per-worker pool is what ecosystem.config.cjs injects: floor(80 / workers) (min 8), unless DB_POOL_MAX was exported in the shell that ran pm2 start / pm2 reload — then that value is used verbatim per worker (4 × 50 = 200 > 100). A DB_POOL_MAX line in .env does not reach the PM2 workers (they already carry the injected value); it only affects CLI scripts and the single-process systemd path. So never run pm2 reload from a shell that has sourced .env ; if in doubt reload with env -u DB_POOL_MAX pm2 reload ecosystem.config.cjs	The cluster cannot exhaust Postgres
P2	Accounts	<code>cd /opt/universe/server && sudo -u universe -H node scripts/manage-user.js list</code>	your real users active; seeded personas (chief.underwriter , retro.manager , underwriter1–4) rotated or deactivated	No demo2026 login remains

#	Check	Command	Expected	Proves
P3	Backup after the switch	<code>ls -lt /var/backups/universe head -2 ; weekly scripts/verify-restore.sh exit 0</code>	a dump newer than the deploy. <code>verify-restore.sh</code> creates and drops a scratch database, so the role in its <code>DATABASE_URL</code> needs <code>CREATEDB</code> — grant it once with <code>sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'</code> (as <code>deploy/README.md</code> step 10 does), or run it with a superuser URL; otherwise it fails with <code>permission denied to create database</code>	Recovery point reflects the migrated schema

7.7 Sign-off summary

- §7.1 C1–C8: one env file, one `DATABASE_URL`, production mode, seeding posture correct, app found that file, nothing baked into the PM2 workers or the PM2 dump, no `server/.env`.
- §7.2 D1–D4: `psql` with the app's URL works and identifies the intended database; role owns it.
- §7.3 R1–R8: boot log, deep health (direct and via nginx), `pg_stat_activity`, user list = `uw_user`, login audited, smoke test `PASSED`.
- §7.4 S1–S7: `Pending (0)`, counts equal, no orphans, no stragglers, last migration as expected, business counts unchanged, key objects really exist.
- §7.5 F1–F4: reference counts as in §5.2, GCC cedants visible, zero fabricated treaties, dropdowns populated.
- §7.6 P1–P3: connection budget, accounts rotated, fresh backup.

Reference — expected seed counts on a freshly migrated database

``` Reference data after seeding: countries 75 currencies 32 brokers 11 reinsurers 25 cedants 40 treaty\_types 11 classes\_of\_business 15 roles 9 Business data (unchanged): contracts 0 quotes 0 claims 0 users 6

On an existing database expect at least these reference counts (plus the Ghana pack and user-added rows); the business rows must be your real, unchanged numbers.

## Sign-off

| Item                                        | Value         |
|---------------------------------------------|---------------|
| Date / time                                 |               |
| Performed by                                |               |
| Deployed commit ( <code>git log -1</code> ) |               |
| <code>migrate:status</code> result          | Pending (0) □ |
| Smoke test (R8)                             | PASSED □      |

| Item                              | Value                                                                            |
|-----------------------------------|----------------------------------------------------------------------------------|
| Orphan / straggler check (S3, S4) | both empty <input type="checkbox"/>                                              |
| Reference counts (F1)             | countries ..... currencies ..... brokers ..... reinsurers ..... cedants<br>..... |
| Business counts (S6)              | contracts ..... quotes ..... claims ..... users .....                            |

Universe Reinsurance Platform — Confidential — Database Link & Alignment Checklist v1.0, September 2026.